

INF-2700: Database systems

Assignment 2

Jens Christian Valen Leynse

J1e040

j1e040@post.uit.no

Oct 16th, 2019

Introduction

The assignment given is to implement some basic elements of database management systems (DBMS) in c. How they work, deal with memory, as well as debugging and using analysis tools and finally understanding the database performance. The assignment is split into 4 tasks. Task 1 is simply understanding the code. Task 2 extending the types of queries the program can handle. Task 3 is implementing binary search as the code uses linear. Task 4 and the final task is comparing with a B⁺-tree.

Requirements:

- Query must be able to handle all comparison searches, namely =, <, <=, >, >= and !=.
- Generating a table of an arbitrary size.
- Performance data for the two search algorithms.

Design & Implementation

Task 2: Extending the types of queries

The function table search in the schema file is responsible for handling the search of the query, so by adding if checks to compare with the operator the different ones can be used as well. Said in other terms, the operator sent is compared with all the available operators and a different function is saved as a variable sent to the searching algorithm.

Task 3: Binary search

The binary search function is to be evaluated together with the linear search function. So, the table search function separates them by a given argument to the function. Both search functions take and give the same data, but the linear function is set as a loop while doing so.

The binary search function takes in the arguments record, schema, the file offset, the operator and the target value for the search. First the variables are defined before the loop. The loop bases its duration on the outer parameters of the search which is defined by the number of records in the schema minus one, and zero which is the lowest possible record

index. The records per block is calculated as the block size minus the header divided by the length of a record.

In the loop the index is always updated as it is placed in the middle of the available records. The block number and the block index are calculated by the number being the index divided by the records per block, and the index beings subtracted to the records per block times the block number. Then the location of the value is found by adding the record length times the block index with the header size of the table with the offset.

Next the page is retrieved and checked if empty given the location in the schema. The value at the current position in the page is retrieved and compared with the target value. If its smaller the lowest parameter is changed to above the index, and if it's larger the opposite happens. When the page value is equal the position is set, and the record retrieved before returning out of the function. If the target value couldn't be found the function return with an error.

To test the binary search function a table needs to be made, and the implementation is placed in the python file `make_table`. This file drops any remaining tables and creates a new one. The implemented loop inserts a tuple of increasing numbers. A quit command is also appended to the commands list before it is all written to a fill file which stores the information.

Further on in the `test_main` file a function to fill and profile the results are implemented. The function that fills the table states the structures record and schema, given the table name, before a loop using a function argument as a stop creates a new record, fills it and appends it to the schema for each iteration.

The profiling function opens the database and retrieves the table by name. The table is then searched, and a different message is put as info for the two different search types. Lastly the pager profiler info is ran, the database closed and the function exited.

The main function now takes; a defined tuple specified as columns, the attributes types and the number of records, and after opening the database creates several test schemas and fills all of them. Lastly the profilins happens as it is used to give info about both the linear and the binary search given different targets. The data found is represented here:

Number of disk seeks/ reads/ writes/ IOs on 100:

	first	half	missing	Random (² / ₃)
Binary search	1/1/0/1	1/2/0/2	1/2/0/2	1/2/0/2
Linear search	3/3/0/3	3/3/0/3	1/2/0/2	2/3/0/3

Number of disk seeks/ reads/ writes/ IOs on 10000:

	first	half	missing	random(² / ₃)
Binary search	8/9/0/9	8/10/0/10	8/10/0/10	9/10/0/10
Linear search	3/165/0/165	3/165/0/165	1/164/0/164	3/165/0/165

Missing: the target value is outside of the parameters letting the function run its course. As the data shows the linear algorithm uses more seeks, writes and reads. Significantly so when using more records during the profiling. And thus, the performance of the binary search is superior.

Task 4: Comparison with B⁺-tree

The B⁺-tree has all the data stored in the leaf nodes creating a balanced tree. This will reduce the height of the tree as more nodes can be placed there.

Storage wise the B⁺-tree must store pointers between the leaf nodes filling any gap. These pointers essentially create a linked list at the leaf level. So, the query doesn't have to be sorted in order since the pointers will point to the next value in the queue, but the index pointing also has to be stored (without value) meaning the B⁺-tree sorted queue has to be larger than the binary queue.

The binary search already knows its size since it's all one queue making the data all accessible on the same level.

As data is only found at the leaf nodes the B⁺-tree it will iterate the indexes first to find the values. The binary search in comparison will compare values at the same level going back and forth.

The sorting notation for the B⁺-tree is " $\log_{1/2} n$ " with the memory "n". While the binary search has the notation " $\log_2 n$ " with the same memory "n". Where n is the record amount. This means that for a database the B⁺-trees will have a better lookup $\log_{1/2} n > 2$.

Discussion

During the final phase of the assignment some design alterations was made to the overall makefile. This would enable it to run the python file to create the table, open the database to insert it and open the shell all in one go. It did work at the start as it enabled the user to specify which SQL command to use and also the option to use more than one. Unfortunately, something went wrong without an easy to find error which would freeze the make command as it was being run consistently every second time the file was run from a fresh computer (after reboot). This made it hard to work around but the problem origin was never found (internal computer issue or makefile issue).

Conclusion

The database management systems have been better understood as the tasks has been completed. The requirements for each task have been implemented in a working manner as the program can run while performing the given tasks.